Verslag labo 3

Opdracht

De opdracht bestaat uit 2 delen. Als eerste moeten we een gegeven afbeelding inverteren met behulp van een kernel. Dit doen we door elke kleurwaarde van de afbeelding gelijk te stellen aan 255 - de huidige kleurwaarde. Hierbij moeten we kijken wat de invloed van het aantal gebruikte threads is op de berekentijd. Als tweede deel gebruiken we de inverterende kernel van deel 1, maar passen we deze aan door een andere bewerking uit te voeren op de R waarden, namelijk $x = (x\%25) \cdot 10, x \in (100, 200)$. Ook hier moeten we kijken wat de invloed is op de berekentijd, ditmaal in verband met thread divergence.

Werkwijze

Als eerste hebben we de code uit de file *image-import.zip* toegevoegd aan onze code, om hiermee een afbeelding te kunnen omzetten naar een grote array van RGB waarden. Hierna hebben we de code voor deel 1 geschreven en hiermee de verschillende tijdsmetingen gedaan. Hierna hebben we de code aangepast om deel 2 van de opdracht te realiseren, en ook hierbij hebben we verschillende tijdsmetingen gedaan.

Code

Na de afbeelding in te lezen definiëren `image_size` en `block_size`. Hier wordt verder het aantal warps mee berekend. Verder worden 2 pointers geïnitialiseerd om plaats op de GPU te reserveren en worden de gegevens overgezet. Aan de kernel wordt de gereserveerde plaats voor het resultaat meegegeven alsook de `image_array` en de `image_size`. De grootte van het grid wordt bepaald door de `block_size` en `THREADS_PER_BLOCK`. In de kernel wordt de absolute thread-id bepaald met behulp van de block-dimension om zo de unieke id te verkrijgen. Met gebruik van een for-loop kunnen we de code een k keer laten overlopen, met als k het aantal iteraties. Binnen deze for-loop bepalen we dan the thread-id met behulp van de bewerking `tid = abs_tid + k * gridDim.x * blockDim.x`. Hierna kijken we of de thread-id kleiner is dan de grootte van de array, om nadien de bewerkingen op de RGB waarden uit te voeren. De invloed van k (ook gekend als het aantal iteraties) zien we in volgende tabel.

Aantal iteraties	1	2	4	16	32	64	128
Verstreken tijd [μs]	1173	1128	1130	1106	1134	1125	1048

Aantal iteraties	256	512	1024	2048	4096	8192	16384
Verstreken tijd [μs]	1051	1057	1096	1147	1104	1115	1100

In de code wordt gebruik gemaakt van warps. Dit is intern de verbinding tussen de blokken en de threads waarbij deze 32 threads omvatten per warp. Dit betekent dat 32 instructies in parallel worden uitgevoerd door de GPU wat

uiteindelijk de timing-prestaties kan verbeteren. Hierbij wordt het wel aangeraden om in veelvouden van 32 te werken mits als de laatste warp minder elementen bevat, deze default waarden van de overige threads neemt en deze dus verloren gaan in efficiëntie. Verder wordt er ook gebruik gemaakt van striding. Hierbij verwijst een zelfde thread naar meerdere plaatsen in de input-array wat mogelijk maakt om zo meer bewerkingen uit te voeren met minder threads. Hier wordt gebruik gemaakt van iteratie die de verwijzing naar de input array maakt om zo de juiste index te kiezen.

Uit de timingresultaten van de tabel kunnen we eigenlijk niet zo veel besluiten. De tijdsduur van de berekeningen bedraagt ongeveer evenveel ongeacht het aantal iteraties, er is eventueel te spreken over een kleine daling in berekentijd.

